

1. Encapsulamento

Definição:

Esconde os detalhes internos do objeto, expondo apenas o que é necessário. Isso é feito usando **modificadores de acesso** (`public`, `private`, `protected`).

Exemplo:

```
public class BankAccount
{
    private decimal _balance; // Atributo privado

    public void Deposit(decimal amount)
    {
        if (amount > 0)
        {
            _balance += amount; // Alteração controlada
        }
    }

    public decimal GetBalance()
    {
        return _balance; // Acesso controlado
    }
}
```

Explicação:

O saldo da conta está encapsulado e não pode ser acessado diretamente, garantindo integridade.

2. Herança

Definição:

Permite criar novas classes que herdam os atributos e métodos de uma classe existente (classe base).

Exemplo:

```
public class Animal
{
    public void Eat()
    {
        Console.WriteLine("This animal eats food.");
    }
}

public class Dog : Animal
{
    public void Bark()
    {
        Console.WriteLine("The dog barks.");
    }
}

// Uso
Dog dog = new Dog();
dog.Eat(); // Herdado da classe Animal
dog.Bark(); // Método específico da classe Dog
```

Explicação:

A classe `Dog` herda o comportamento de `Animal` e adiciona funcionalidades próprias.

3. Polimorfismo

Definição:

Permite que métodos ou objetos tenham comportamentos diferentes, dependendo do contexto.

Exemplo:

```
public class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("The animal makes a sound.");
    }
}

public class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("The dog barks.");
    }
}

public class Cat : Animal
{
    public override void Speak()
    {
        Console.WriteLine("The cat meows.");
    }
}

// Uso
Animal animal = new Dog();
animal.Speak(); // "The dog barks."

animal = new Cat();
animal.Speak(); // "The cat meows."
```

Explicação:

O mesmo método (`Speak`) tem diferentes implementações dependendo do tipo do objeto (`Dog` ou `Cat`).

4. Abstração

Definição:

Destaca os comportamentos essenciais e oculta os detalhes. Feito usando classes abstratas ou interfaces.

Exemplo:

```
public abstract class Shape
{
    public abstract double CalculateArea(); // Definição abstrata
}

public class Circle : Shape
{
    public double Radius { get; set; }

    public override double CalculateArea()
    {
        return Math.PI * Radius * Radius; // Implementação específica
    }
}

public class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }

    public override double CalculateArea()
    {
        return Width * Height; // Implementação específica
    }
}

// Uso
Shape circle = new Circle { Radius = 5 };
Console.WriteLine(circle.CalculateArea()); // Área do círculo

Shape rectangle = new Rectangle { Width = 4, Height = 6 };
Console.WriteLine(rectangle.CalculateArea()); // Área do retângulo
```

Explicação:

A classe `Shape` abstrai o conceito de forma. Classes concretas (`Circle`, `Rectangle`) implementam os detalhes.

Vantagens da OOP

1. **Reutilização de Código:** A herança permite criar novas classes sem duplicação de código.
2. **Modularidade:** Objetos encapsulam dados e comportamentos relacionados.
3. **Facilidade de Manutenção:** Alterações em uma classe afetam apenas o que está relacionado a ela.
4. **Escalabilidade:** Novas funcionalidades podem ser adicionadas facilmente.